Graph Exploitation Lab

MDS

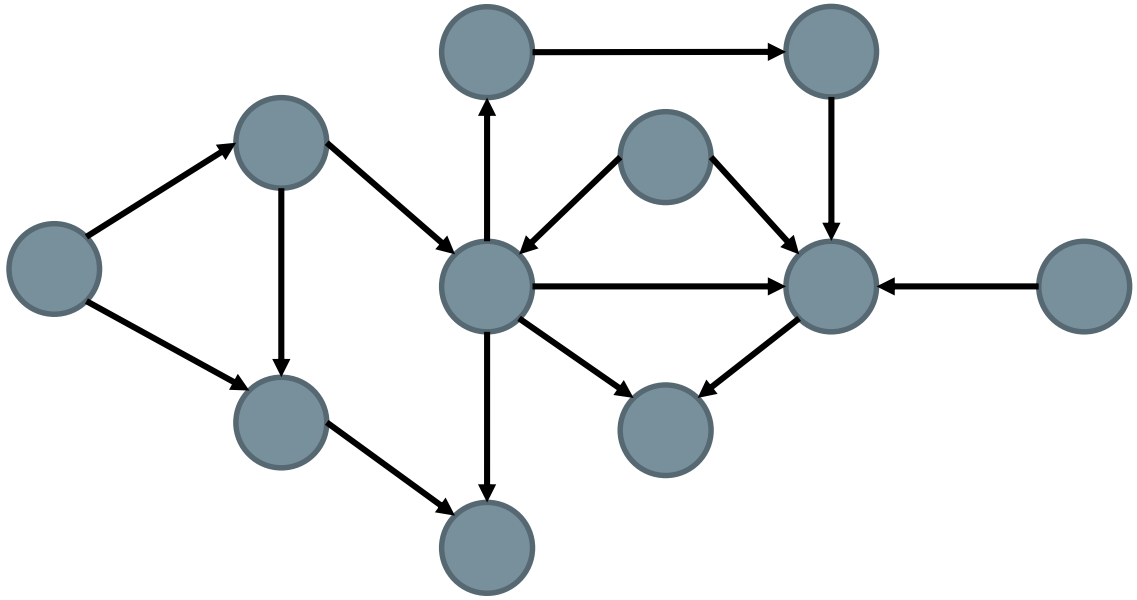
Graph Exploitation Lab

In this lab, we are going to work with graph data, exploiting their structure by using **Graph Neural Networks** and **Knowledge Graph Embeddings**. In particular, we will see:

- How to train **KGE** models.
- How to **interpret** the resulting embeddings.
- How to **create** Graph Neural Networks.
- How graph information can **improve classical ML systems**.
- **Characteristics** of Graph Neural Networks.
- How Graph Neural Networks can **benefit from** Knowledge Graph Embeddings.

Lab Context

In this lab we will first explore **KGEs**, and we will learn how they are trained and the effect different **training configurations** have on them.



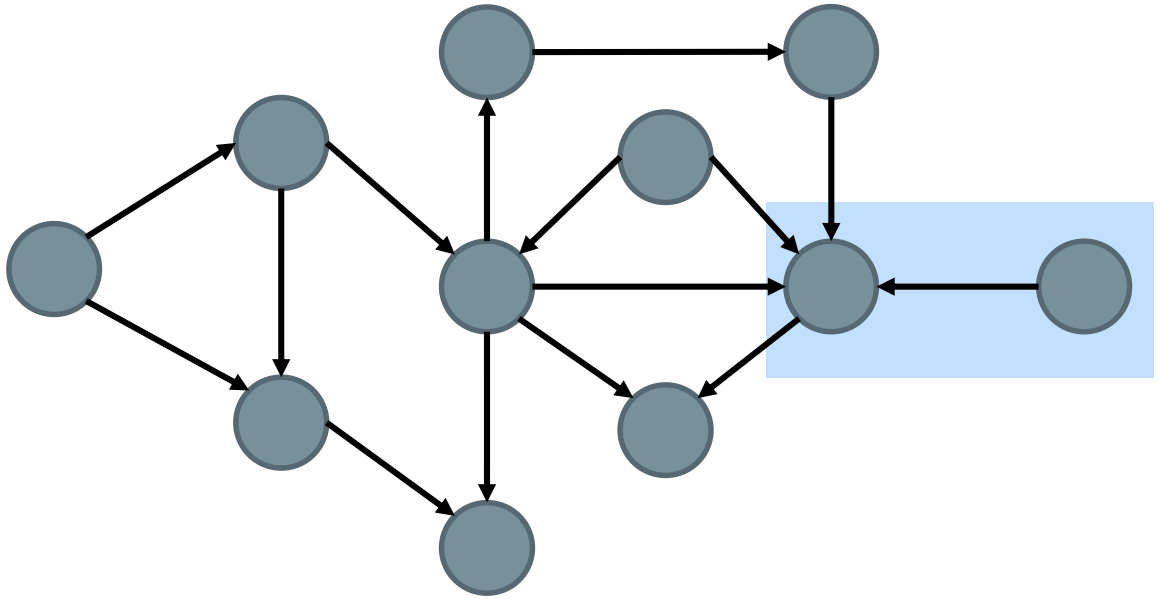
PubMed

KGE Learning: Parameters

- Model
- Margin
- Number of Negatives samples per Positive Sample
- Embedding Dimension
- Learning Rate
- Epochs

KGE Learning: Dataset Partitions

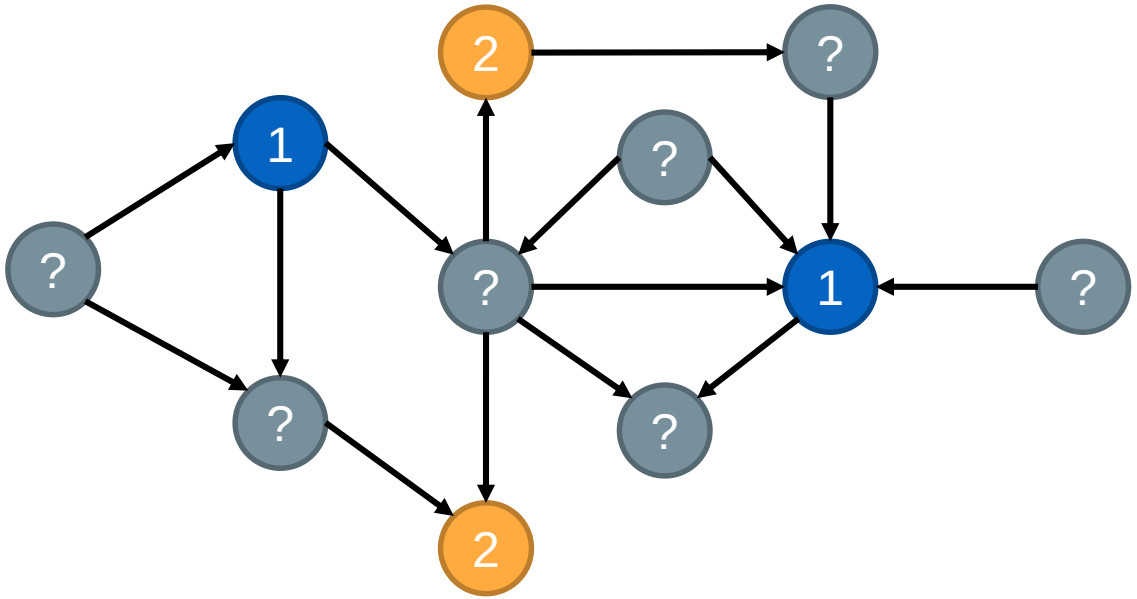
The **train/test/validation partitions** used for training KGEs need to be carefully made, as a test prediction will not work if we have not even created an embedding for one of the entities present in the triple. Therefore, we must ensure that all the different entities and relations are present at least once in the training dataset. This process is know as **stratification**.



PubMed

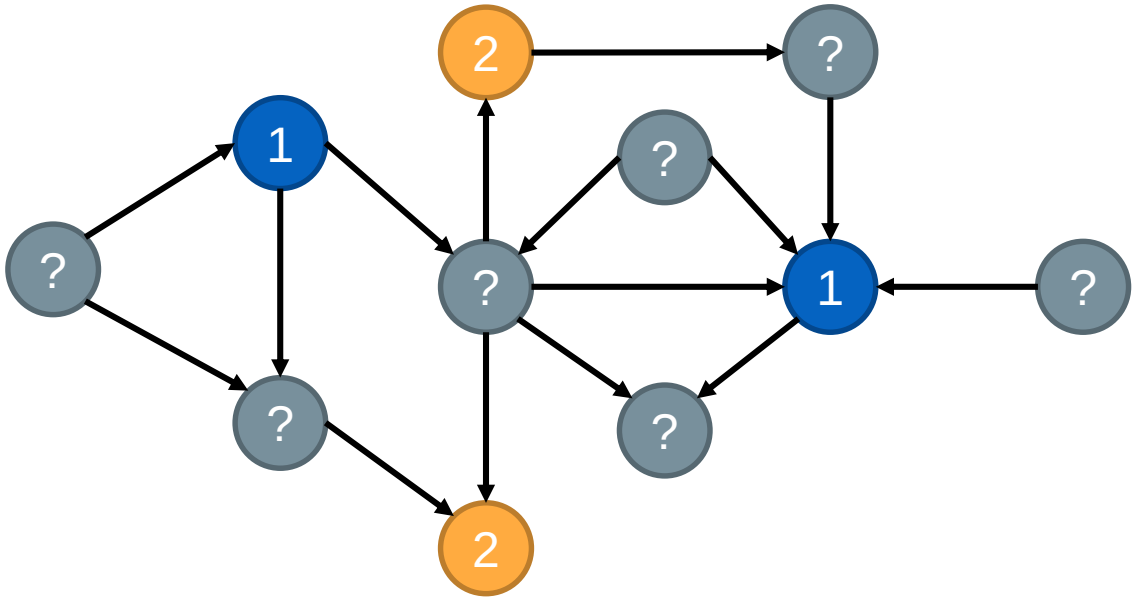
Lab Context

Then, we will enrich the **PubMed** graph with additional information. Concretely, each node (i.e., Paper) will be given a **feature vector** that has been computed from its abstract, and also some of the nodes will be given **labels** according to the type of diabetes they target.



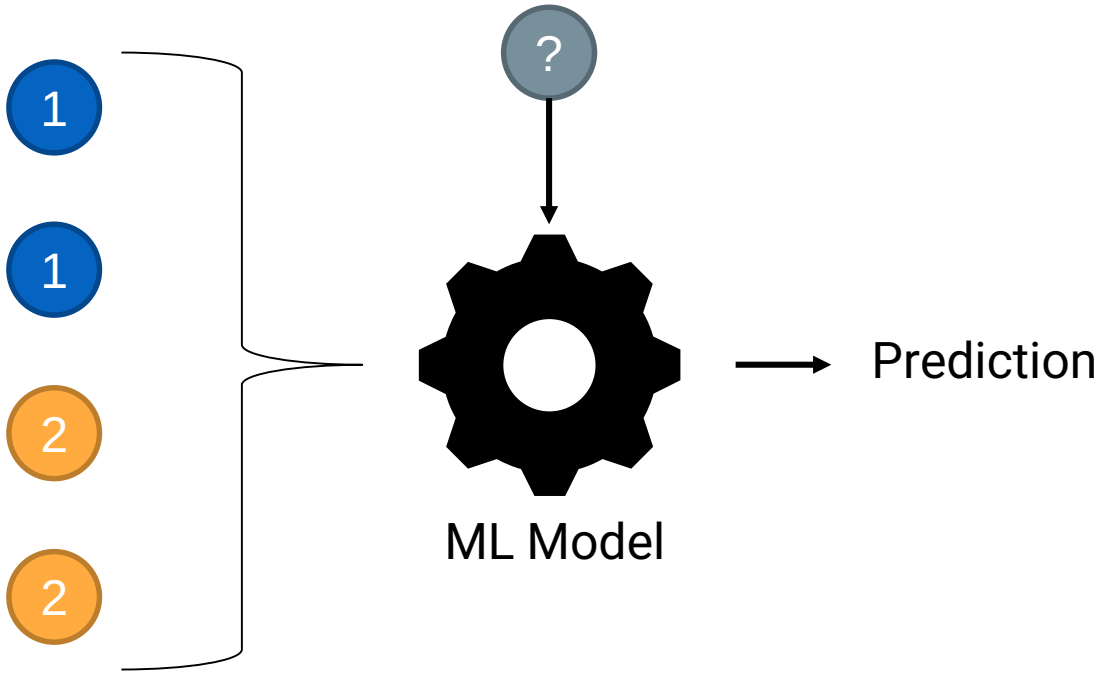
Lab Context

With that, we will also address a **semi-supervised task** with GNNs. This is, we have a lot of data points, but just a small fraction of them are **labeled**. This small sized training dataset makes it difficult to create powerful predictive models. Nevertheless, we will use the **connections of the graph** to propagate relevant information **even though** we do not have labels.



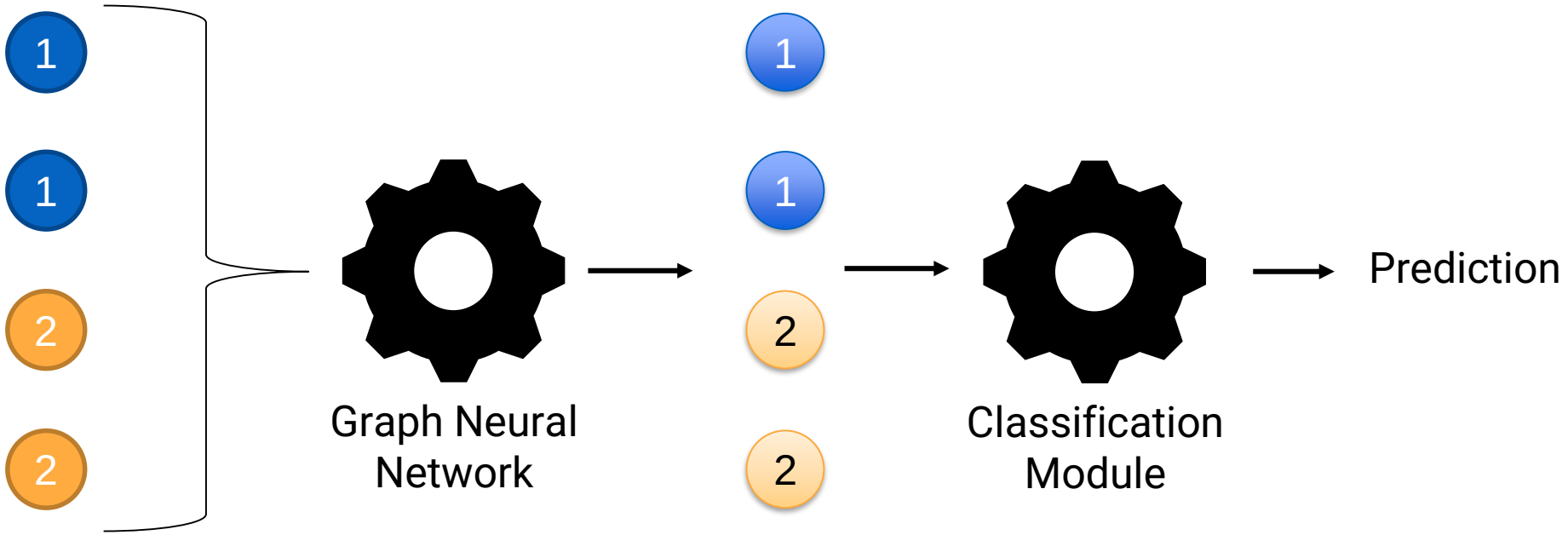
Lab Context

First, we will try to solve the problem using **typical ML models**. Therefore, we will use the **feature vectors** of the labeled points to train a classifier.



Lab Context

Then, we will solve the problem using **Graph Neural Networks**. With that, we will first transform the **feature vectors** of the labeled points based on their neighbors before creating a classifier.



How to Build a Graph Neural Network

To build our Graph Neural Network, we will use the **PyTorch** library. PyTorch is an open-source library that is typically used to create **deep learning** models. It follows a **modular** approach, so we can easily and intuitively built our models by joining together different deep learning **components**. We can image it as putting together different **building blocks** (like LEGO!). Therefore, we will:

1. **Define/create** our building blocks.
2. **Join** them together in the desired order.
3. **Pass the data** through them to train the learnable parameters.

The only consideration is that the **input** and **output** dimension of the blocks that are joined together must match.

How to Build a Graph Neural Network

class MyModel(nn.Module):
def __init__(self):

Define the building blocks

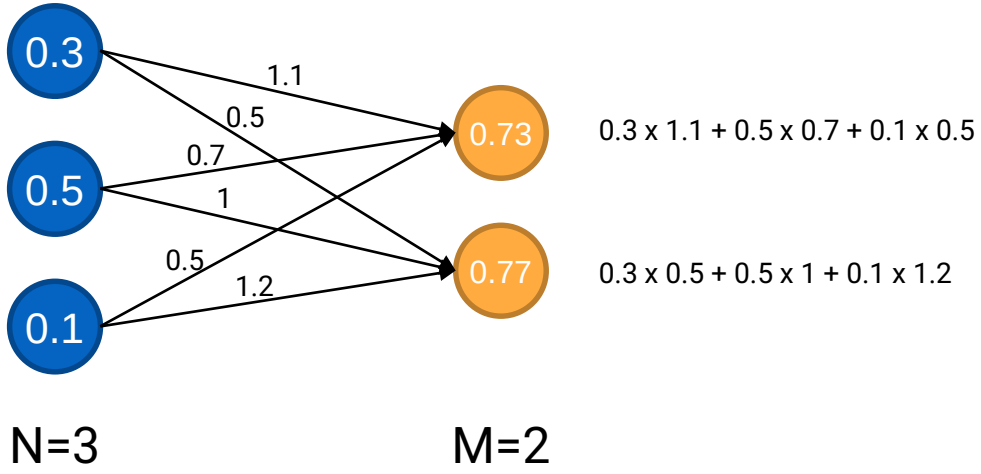
def forward(self, x, edges):

Put the building blocks together

return x

Building Blocks: Linear Layer

The first building block are **Linear Layers**. This block receives a vector of **size N** and outputs a vector of **size M** by **linearly combining** the elements of the input vector.




```
linear_layer = nn.Linear(N, M)
```

Building Blocks: Graph Neural Networks

In **Graph Neural Networks** we again we start with a vector of **size N** and end with a vector of **size M** through more complex computations that take into account the **graph structure**. In PyTorch, there exist **already defined GNN blocks** that we can simply add to our model: GCNConv, SageConv, etc. You can find the complete list here: https://pytorch-geometric.readthedocs.io/en/latest/cheatsheet/gnn_cheatsheet.html

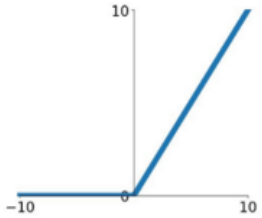
```
from torch_geometric.nn import GCNConv

gnn_layer = GCNConv(N, M)
```

Building Blocks: Activation Functions

In Deep Learning models, activation functions are used to introduce **non-linearities**. This is, combining only linear functions is not effective to introduce **depth** in the network, as one could find a single linear function doing so. This functions **do not change** the size of the input vector, they just transform its value:

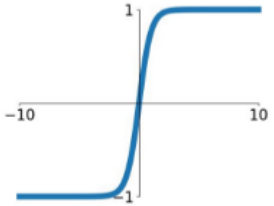
ReLU

$\max(0, x)$



tanh

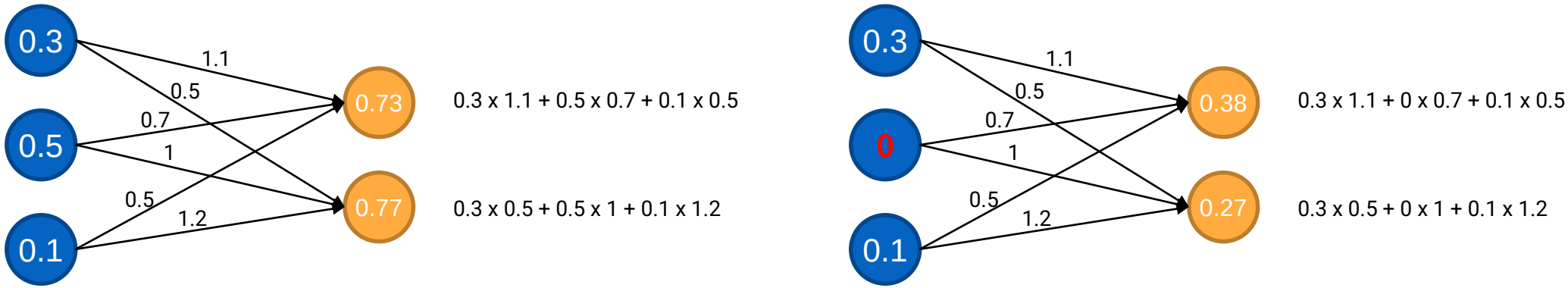
$\tanh(x)$



activation_layer= nn.functional.relu()

Building Blocks: Dropout Layers

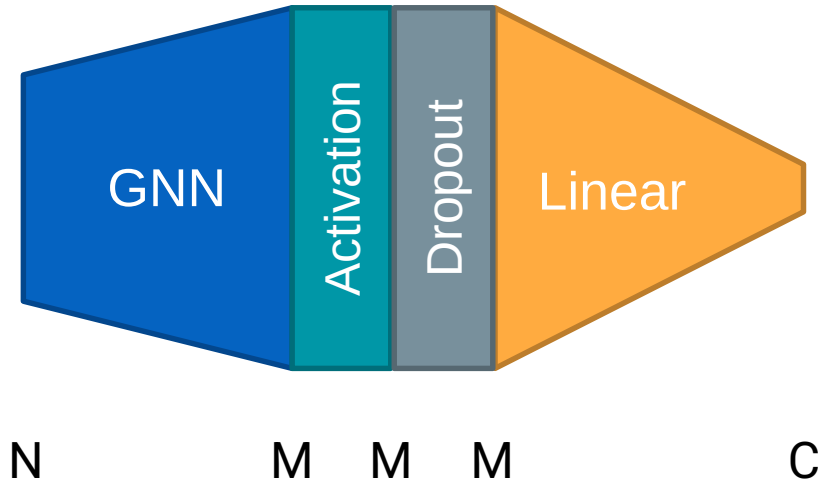
In Deep Learning models, a Dropout layer used to prevent **overfitting** by **randomly** setting a fraction of input values to **zero** during training. This functions **do not change** the size of the input vector.




```
dropout_layer = nn.Dropout(p=0.5)
```

Simple Example

Given a graph where the nodes have a **feature size of N**, we want to classify them in **C classes**.




```
class MyModel(nn.Module):
    def __init__(self,N,M,C):
        self.conv = GCNConv(N,M)
        self.linear = nn.Linear(M,C)

    def forward(self, x, edges):
        x = self.conv(x,edges)
        x = nn.functional.relu(x)
        x = F.dropout(x,p=0.5)
        x = linear(x)
    return x
```

Questions?